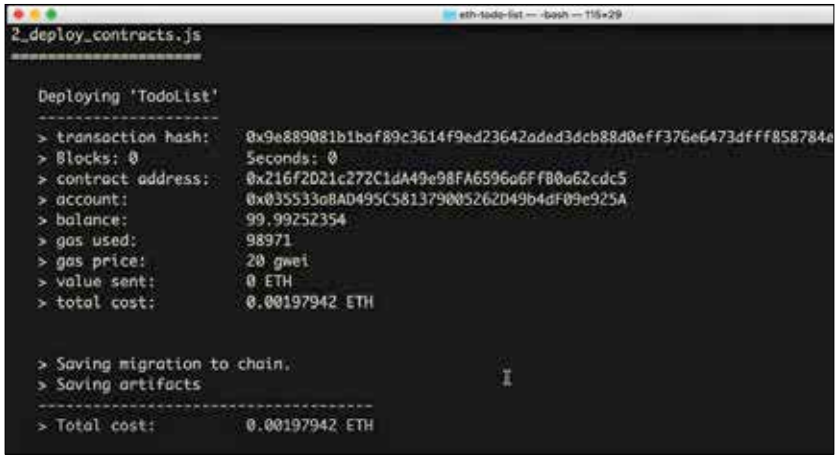
Air taxi service in South Korea
Joby Aviation to launch air taxi service in South Korea in partnership with SK Telecom.
https://dgit.in/mar22-43

Apple on AirPods Max
Apple looks to replace AirPods Max's digital crown with touch controls.
https://dgit.in/mar22-44

Well, whenever you deploy a new block of code to the blockchain, you are changing the structure or schema of the blockchain. Putting this smart contract on the blockchain will update the state of the blockchain, similar to how you add data to a database in SQL. To facilitate this update, you need a migration. Think of it as a carrier of code from your smart contract file to the blockchain network. Your code inside 2_deploy_contracts.js should look like the image below:

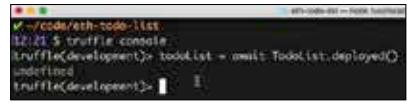
This file is an abstraction of the contents of the smart contract written inside TodoList.sol, and that code snippet will be deployed on the blockchain by running this file. Run the migration using:

Remember that running each smart contract requires a certain amount of ETH currency. After every migration, you will see a log of the costs incurred, in this manner:



RETRIEVING YOUR SMART CONTRACT

To see your smart contract on the blockchain and retrieve it, you will have to open the truffle console, which you can do by typing truffle console in your command line. This will lead you to the truffle development environment, which is similar to the dev console in Chrome where you can write javascript. In this console, you can write in solidity to retrieve a copy of your smart contract in the following manner:



A copy of your smart contract has been saved in the variable todoList.

The await keyword signifies an asynchronous action. Interactions with smart contracts need to be done in an asynchronous manner. With truffle 5, the "await" keyword lets you employ the async/await pattern to perform these interactions.

INSIDE A SMART CONTRACT

You can view the contents of the smart contract in your console by simply typing the variable name. A smart contract can be loosely compared to a javascript object. Basically, the contents of the smart contract can be viewed in the console using dot notation to fetch a specific content. For example, if you want to see the address of the smart contract, you can use: todoList.address and press enter in your console. This will give you the address of the locally deployed smart contract.

Similarly, we had created a variable inside the smart contract and assigned it to 0. To view that variable, you can use todoList.taskCount(), which will return <BN: 0>, associating to the number of tasks present in our to-do list so far. Truffle stores this value as an object 'BN', or Big Number. To retrieve this variable as a number, we have to perform a conversion:

Congratulations! If you were able to get this far, you have officially created a smart contract, used truffle to launch the smart contract on a (local) blockchain (provided by Ganache), and communicated with this blockchain using async/await pattern. While this provides insight into local development, what about connecting with the global blockchain network?

BLOCKCHAIN ON THE WEB

Similar concepts, but different sets of tools are used to execute smart contracts on the web. Similarly, existence on a web that can be accessed by everyone leads to some level of centralisation of blockchain. But to access currencies and wallets, our smart contracts need to interact with some real world data. Hence, in implementation, blockchain on the web cannot be decentralised completely. Hence, the contracts coded to perform transactions on the web are called "hybrid smart contracts".

These smart contracts use a concept called "oracle", which is an entity that allows a decentralised network to